FailureOps: Counterfactual Failure Attribution in Quantum Computing

Anonymous Author(s)
Anonymous Institution
Anonymous, Anonymous
anonymous@anonymous

Abstract

Quantum error correction (QEC) protects logical computations by encoding them across many physical qubits and continuously decoding syndrome data. Current practice evaluates QEC performance through aggregate logical error rates and static error-type labels. These metrics describe where errors occur but cannot answer a systems question: which controllable factors, if changed, would alter the observed logical failure behavior of a given execution?

We present FailureOps, a paired counterfactual attribution methodology that answers this question. FailureOps evaluates the same shot records under a baseline and an intervened configuration, preserving shot identity, seed, and detector-event trace. It measures rescued and induced logical-failure transitions and reports which interventions change failure behavior on the same physical execution records. We instantiate FailureOps on public real QEC detector records from Google's RL QEC and qec3v5 datasets, with interventions spanning decoder-pathway switching, decoder-prior families, and measured-runtime deadline replay.

Across 40 real data conditions, the main decoder-pathway intervention rescues logical failures in every observed condition (40/40 net-rescuing, 39/40 individually significant after scope-wise Holm correction at $\alpha = 0.05$). Paired counterfactual estimation reduces estimator standard deviation by a factor of 1.75 relative to unpaired resampling, a necessary condition for distinguishing rescued from induced transitions. A corpus-level decoder-prior study across 5,724 prior variants yields 85.2% of confidence intervals excluding zero. The decoder-pathway result externally replicates on the qec3v5 surface-code dataset (23/26 Holm-significant), and the signal remains broadly net-rescuing on an expanded 496-condition Google RL QEC v2 corpus. Measured decoder service time can be closed into a paired deadline intervention exhibiting a sharp threshold near 5 μs.

FailureOps does not propose a new decoder, code, or threshold-estimation method. It provides a paired debugging and attribution interface for QEC-protected logical executions, treating logical failure as an intervention-sensitive system event rather than a static error statistic. All evaluation uses public real detector records, and the full pipeline is reproducible from open data.

ACM Reference Format:
Anonymous Author(s). 2027. FailureOps: Counterfactual Failure Attribution in Quantum Computing. In *Proceedings of European Conference on Computer Systems (EuroSys '27).* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Quantum error correction is entering its systems era. As experimental platforms scale toward fault-tolerant operation, the systems community must confront questions that go beyond physics-level code performance: how should a QEC decoder be scheduled under real-time constraints? Which decoding strategy is appropriate for a given workload? When does idle time between syndrome cycles become a dominant failure source? Answering these questions requires more than measuring aggregate logical error rates. It requires knowing which controllable factors, if changed, would alter the logical failure behavior of a given protected execution.

Current QEC evaluation practice is not designed to provide this kind of attribution. The standard metrics—logical error rate, baseline logical failure rate (LFR), syndrome-detector burden, error-type classification—are descriptive. They can rank conditions by difficulty and label failures by suspected physical origin, but they do not answer the counterfactual question: would this specific execution still have failed if the decoder, the prior, or the runtime budget had been different? A high detector-event count on a failed shot does not guarantee that a different decoder would have corrected it. An idle-error label on a syndrome round does not tell you whether shortening the inter-cycle gap would have changed

EuroSys '27, Rabat, Morocco

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

the logical outcome. Attributing failure behavior to intervenable factors requires an experimental design that isolates those factors while holding everything else fixed. This is the gap that FailureOps fills.

FailureOps is a paired counterfactual attribution methodology for QEC-protected logical circuit executions. Given a set of shot-level detector records—the same data that a real QEC system produces during operation—FailureOps runs the same shots through a baseline configuration and one or more intervened configurations, preserving shot identity, random seed, detector-event trace, and logical workload. It then measures two quantities on discordant shot pairs: rescued failures (shots that fail under the baseline but succeed under the intervention) and induced failures (shots that succeed under the baseline but fail under the intervention). The core output is a sensitivity profile: which intervention, applied to which class of executions, changes logical failure behavior the most, and in which direction? The attribution is tied to the intervention, not to a static label.

We instantiate FailureOps on public real QEC detector records, using interventions that span three families: decoder-pathway switching (changing the decoder backend or the prior distribution that feeds it), decoder-prior variation (sweeping across thousands of prior configurations within a fixed decoder family), and measured-runtime deadline replay (applying decoder service-time budgets measured on real records). We evaluate across three public QEC detector-record datasets that support per-shot paired analysis: the original Google RL QEC detector-record release, the expanded Google RL QEC v2 corpus, and the Google qec3v5 surface-code scaling dataset. An additional IBM aggregate hardware-validation dataset provides independent sanity-check evidence but does not contain per-shot detector records and is analyzed separately.

We report six main results:

The main decoder-pathway intervention—switching from a correlated-matching decoder to a tesseract decoder on the same shot records, both using the same SI1000 prior—rescues logical failures in all 40 observed real-data conditions, with 39 of 40 individually significant after scope-wise Holm correction. No condition shows net-induction.

Pairing is methodologically necessary, not statistically convenient. Across all observed conditions on both the original and expanded corpora, unpaired bootstrap resampling inflates estimator standard deviation by a factor of 1.5–3.1 relative to paired resampling. Without pairing, the rescued and induced transition semantics that distinguish FailureOps attribution from aggregate LFR comparison are lost.

A corpus-level decoder-prior study across 5,724 prior variants (19.1 million paired shots) shows 85.2% of confidence intervals excluding zero, with mean paired delta remaining negative in all three prior families. The decoder-prior effect is not a single-condition anecdote but a replicable intervention-family phenomenon.

Measured decoder service time (mean 4.65 μs) produces a sharp deadline-intervention threshold: a 4 μs budget causes 93.6% of corrections to arrive late and induces a paired delta LFR of +0.307, while a 7 μs budget eliminates the effect entirely. This is a measured-service-time replay closure, not a claim of live control-stack timing.

The decoder-pathway result externally replicates on the independent qec3v5 surface-code dataset with a different decoder baseline (PyMatching), yielding 23 of 26 Holm-significant conditions and preserving the sign of the effect across both logical bases.

The signal is not confined to a narrow condition matrix. An expanded same-family corpus of 496 Google RL QEC v2 conditions shows 461 net-rescuing conditions and a mean paired delta LFR of -0.014, with no subgroup reversal across code family, control mode, or logical basis.

The contributions of this paper are:

(1) A paired counterfactual attribution methodology that treats logical failure as an intervention-sensitive system event rather than a static error statistic.
(2) A public-data instantiation of the methodology across decoder-pathway, decoder-prior, and runtime-replay intervention families.
(3) Empirical evidence, drawn from hundreds to thousands of real QEC conditions and millions of paired shots, that paired counterfactual attribution exposes intervention-sensitive failure behavior that descriptive baselines cannot capture.
(4) A reproducibility-first artifact: the full pipeline runs on public detector records, produces a canonical set of paper-facing output tables and figures, and is designed for reviewer verification.

FailureOps does not propose a new decoder, code family, or threshold-estimation method. It is not a simulator. It is an attribution layer that sits on top of existing QEC pipelines and answers a question that the systems community will need as QEC-protected quantum computation becomes a real engineering discipline: given a protected execution, what controllable factor, if changed, would have altered its logical failure behavior?

The remainder of this paper is organized as follows. Section 2 provides background on QEC and logical failure measurement. Section 3 presents the FailureOps design and paired counterfactual framework. Section 4 describes the implementation pipeline and the intervention registry. Section 5 presents the evaluation across the datasets and three intervention families. Section 6 discusses limitations, scope boundaries, and the relationship between FailureOps and future live-system deployment. Section 7 surveys related work

in QEC evaluation, systems attribution, and counterfactual methods. Section 8 concludes.

2 Background and Motivation

2.1 Quantum Error Correction in Brief

A logical qubit is protected by encoding it across many physical qubits and repeatedly measuring stabilizer operators that reveal the presence of errors without collapsing the logical state. Each measurement round produces a set of detector events—comparisons between consecutive stabilizer measurements that signal where a physical error may have occurred. A decoder consumes this stream of detector events and produces a correction prediction: which physical qubits should be flipped to restore the logical state.

The detector-event record is the observable output of a QEC system during operation. It contains, for each shot (a single logical-circuit execution), the full temporal sequence of detector firings across all measurement rounds. This record is what a real QEC control system would log during live operation, and it is the input to every decoder that must produce a correction in real time.

2.2 Logical Failure and Its Measurement

A logical failure occurs when the decoder's correction prediction, combined with the actual physical errors on the qubits, results in a net logical error—the logical state is not what it should have been. The logical failure rate (LFR) is the fraction of shots in which this occurs. It is the primary metric by which QEC performance is evaluated.

Current practice measures LFR at two levels. The aggregate level reports the fraction of failing shots across an experiment condition, typically broken down by code distance, cycle count, basis, and control mode. The diagnostic level labels individual failures by suspected physical origin: a data-qubit error, a measurement error, an idle error, or a decoder mistake. Both levels are valuable. Aggregate LFR tells you how well a code is performing. Error-type labels tell you where the trouble might be.

What neither level provides is an answer to the counterfactual question: if a specific controllable factor had been different—the decoder algorithm, the prior distribution, the runtime deadline budget—would this particular shot still have failed? A shot with a high detector-event count under a data-error label might or might not have been correctable by a different decoder. A shot labeled as an idle-error failure might or might not have survived a tighter inter-cycle schedule. Aggregate LFR and error labels describe what happened; they do not attribute logical failure to intervenable factors.

2.3 The Attribution Gap

The gap between description and attribution matters for systems design. Consider three scenarios:

A QEC engineer must choose between two decoder backends. Both achieve similar aggregate LFR on a benchmark suite. The choice would be arbitrary—unless one rescues more failures than the other on the same shots, and the rescue pattern is concentrated in a particular cycle regime or error-type profile. Knowing which decoder to deploy requires knowing which decoder changes failure behavior on which shots, not which decoder has the lower average error rate.

A runtime system designer must set a decoder deadline. A shorter deadline reduces cycle time but risks late corrections; a longer deadline is safer but costs throughput. The optimal deadline depends on how many shots would be rescued or induced at each budget level. Aggregate LFR sensitivity to cycle count does not answer this question, because it does not distinguish the effect of the deadline from the effect of longer memory exposure.

A researcher proposes a new prior distribution for a decoder. Testing the new prior on a fresh set of shots might show a lower LFR, but this could be due to sampling variation or shot-level differences unrelated to the prior. Without comparing the new prior and the baseline prior on the same shots, it is impossible to attribute the LFR difference to the prior change rather than to shot-level variation.

In all three cases, the missing ingredient is a paired counterfactual: the same shot records, evaluated under both the baseline and the intervened configuration, with the outcome difference attributed to the intervention. This is the design that FailureOps provides.

3 FailureOps Design

3.1 Paired Counterfactual Framework

The core design principle of FailureOps is that attribution must be tied to an intervention on the same execution records. The unit of analysis is a paired shot: a single logical-circuit execution whose detector-event record is processed twice—once under a baseline configuration and once under an intervened configuration—preserving shot identity, random seed, detector-event trace, and logical workload. The only difference between the two passes is the intervention.

Formally, for a shot i with detector record d_i, the baseline configuration C_0 produces a correction prediction $c_{0,i}$ and a logical outcome $o_{0,i} \in \{0, 1\}$ (0 = success, 1 = failure). The intervened configuration C_1 produces $c_{1,i}$ and $o_{1,i}$ on the same record d_i. The paired transition is one of four cases:

- **Rescued failure**: $o_{0,i} = 1$, $o_{1,i} = 0$. The shot failed under the baseline but succeeded under the intervention.

- **Induced failure**: $o_{0,i} = 0$, $o_{1,i} = 1$. The shot succeeded under the baseline but failed under the intervention.
- **Unchanged failure**: $o_{0,i} = 1$, $o_{1,i} = 1$. The shot failed under both configurations.
- **Unchanged success**: $o_{0,i} = 0$, $o_{1,i} = 0$. The shot succeeded under both configurations.

The paired delta logical-failure rate is defined as $\Delta = (R - I)/N$, where R is the rescued failure count, I is the induced failure count, and N is the total number of paired shots. A negative Δ indicates that the intervention rescues more failures than it induces; a positive Δ indicates the reverse.

This design makes the counterfactual explicit. The question "does this shot still fail under the intervention?" is answered by comparing $o_{0,i}$ and $o_{1,i}$. The rescued and induced counts are not inferred from aggregate LFR differences across independent shot batches; they are directly counted over discordant pairs on the same detector records.

3.2 Sensitivity Profile

The primary output of FailureOps is a sensitivity profile: for a given intervention family, which conditions exhibit the largest paired effect, in which direction, and with what statistical confidence? A sensitivity profile is richer than an aggregate LFR comparison because it preserves the condition-level structure. Two interventions might produce the same mean delta LFR but very different sensitivity profiles: one might rescue failures uniformly across all conditions, while another might strongly rescue long-cycle conditions but induce failures on short-cycle ones. The sensitivity profile exposes this structure.

A sensitivity profile can be aggregated along any dimension present in the condition metadata: code family, control mode, logical basis, cycle count, or detector-burden characteristics. This allows FailureOps to answer not only "which intervention changes failure behavior most?" but also "on which classes of executions is the intervention most effective?"

3.3 Intervention Families

FailureOps defines interventions as a controlled change to a single component of the QEC processing pipeline. We structure interventions into families, where each family varies one dimension while holding the rest of the pipeline fixed.

Decoder-pathway interventions change the decoder algorithm or the prior distribution that feeds it. The primary decoder-pathway intervention in this paper switches from a correlated-matching decoder to a tesseract decoder, both using the same SI1000 prior (a uniform prior that assumes all error mechanisms are equally likely). This is a natural systems question: given the same detector records, does switching to a different decoder backend change which shots succeed and which fail? A complementary prior-switching intervention—keeping the tesseract decoder fixed and switching from SI1000 to frequency-calibrated prior—is evaluated in the expanded corpus (Section 5.6) and decoder-prior study (Section 5.7).

Decoder-prior interventions sweep across multiple prior configurations within a fixed decoder family. This family turns the binary prior switch into a design-space exploration: given a decoder backend, which prior configuration minimizes induced failures? The corpus-level evaluation across 5,724 prior variants answers this at scale.

Runtime-replay interventions apply a deadline budget to decoder corrections based on measured service time. If the recorded service time for a shot exceeds the budget, the correction is treated as late and dropped. This family connects the static paired framework to a dynamic runtime constraint without claiming live hardware feedback.

An intervention family is specified by: a baseline configuration, an intervened configuration, the component of the pipeline being varied, the condition matrix over which the intervention is evaluated, and the shot-records that serve as input.

3.4 Statistical Framework

For each condition within an intervention family, we test the null hypothesis that the rescued and induced failure counts are equal using an exact McNemar test over discordant pairs. The test statistic is the two-sided binomial probability of observing at least $|R - I|$ discordant pairs in either direction, given $R + I$ total discordant pairs and a null probability of 0.5 for each direction.

Because we evaluate multiple conditions within an intervention family, we apply Holm-Bonferroni correction to control the family-wise error rate at $\alpha = 0.05$. Critically, we apply Holm correction within each analysis scope separately: the real decoder-pathway scope, the real decoder-prior scope, the measured-runtime-deadline scope, the external qec3v5 scope, and the Google RL QEC v2 scope. This scoped correction prevents the main real-record decoder-pathway claim from being penalized by thousands of unrelated prior-variant tests, while still controlling the error rate within each claim family.

We report both the Holm-adjusted significance and the raw effect direction for every condition. A condition that is net-rescuing but not individually significant still contributes to the sign-consistency analysis (the fraction of conditions with negative delta). This dual reporting—significance for individual claims, sign consistency for pattern-level evidence—ensures that the evaluation does not discard informative conditions merely because they fall below a corrected threshold.

3.5 What FailureOps Is Not

FailureOps is not a new QEC decoder, nor a new quantum error-correcting code, nor a threshold-estimation framework. It is not a simulator. It is an attribution layer that operates on detector records produced by whatever QEC pipeline is already in place. It does not modify the decoder, the code, or the physical error model. It answers a question that these components do not address: given a protected execution, what controllable factor, if changed, would have altered its logical failure behavior?

4 Implementation

The FailureOps pipeline is implemented as a set of standalone Python scripts that process public QEC detector records and produce the paper-facing output tables and figures. The implementation is structured as a layered pipeline with four stages: data ingestion, intervention execution, evidence aggregation, and digest generation.

4.1 Data Ingestion

The pipeline ingests per-shot detector records from three public sources that support the full paired pipeline: the original Google RL QEC release, the Google RL QEC v2 expanded corpus, and the Google qec3v5 2022 surface-code scaling dataset. Each dataset is stored in its native format under `data/raw/`. A set of dataset-specific ingestion scripts normalizes the records into a common schema with fields for shot ID, detector-event sequence, logical workload configuration, control mode, basis, and cycle count. The normalized records are stored as CSV files under `data/results/` and serve as the single input surface for all downstream interventions. An additional DAQEC/IBM aggregate hardware-validation dataset provides independent external sanity-check reference points but does not contain per-shot detector records and is analyzed separately.

4.2 Intervention Registry

Interventions are defined declaratively. Each intervention specifies a baseline configuration, an intervened configuration, the pipeline component being varied, and the condition matrix over which to evaluate. The baseline and intervened configurations reference decoder backends (e.g., PyMatching, correlated matching, tesseract), prior distributions (e.g., SI1000, frequency-calibrated, dem_correlations, dem_rl_isolated), and runtime parameters (e.g., deadline budgets in microseconds). The intervention registry is versioned and hash-identified so that every output artifact can be traced to the exact intervention specification that produced it.

4.3 Execution

For each condition in an intervention family, the pipeline iterates over the paired shot records and executes both the baseline and the intervened configuration on each shot's detector-event trace. The executions are independent across shots and conditions and can be parallelized. The pipeline records, for each shot, the baseline correction, the intervened correction, the baseline logical outcome, the intervened logical outcome, and the paired transition class (rescued, induced, unchanged-failure, unchanged-success). These per-shot transition records are the atomic unit of FailureOps evidence and are preserved for all downstream analyses.

4.4 Output and Reproducibility

The pipeline produces a canonical set of paper-facing outputs: per-intervention-family significance tables with McNemar exact p-values and Holm-adjusted significance, baseline-comparison summaries, robustness-check tables, a claim-audit table that maps each paper claim to its supporting evidence and scope boundary, and a reviewer-facing digest that compresses the main results into a single compact table. The full pipeline is runnable from a single command (`python scripts/31_run_p10_eurosys_main_evidence.py`) followed by the digest builder (`python scripts/37_build_p10_eurosys_digest.`
All input data is public and all intermediate artifacts are human-readable CSV files.

5 Evaluation

We evaluate FailureOps across three public per-shot QEC detector-record datasets (plus an IBM aggregate sanity-check dataset) and three intervention families. The main real-record evaluation uses the Google RL QEC detector-record release (40 conditions, 400,000 paired shots), with decoder-pathway intervention as the primary treatment. The pairing-necessity analysis uses bootstrap resampling over paired and unpaired estimators on the same conditions. The baseline comparison evaluates four conventional metrics—plain LFR, static detector burden, plain LFR delta, and unpaired bootstrap—against FailureOps paired sensitivity. The runtime-replay evaluation applies measured decoder service-time traces to a paired deadline intervention. The external and expanded evidence section covers the qec3v5 surface-code replication (§5.5), the Google RL QEC v2 496-condition expansion (§5.6), and the corpus-level decoder-prior study across 5,724 prior variants (§5.7). All statistical tests use exact McNemar tests over discordant rescued and induced transition counts, with scope-wise Holm correction applied at $\alpha = 0.05$ within each analysis scope.

5.1 Main Real-Record Result

The primary FailureOps intervention switches the decoder backend from correlated matching to tesseract, with both configurations operating on the same shot-level detector records and using the same SI1000 prior (a uniform prior that assumes all error mechanisms are equally likely). The tesseract decoder employs a different matching algorithm, and the systems question is: given the same detector records and the same prior, does switching to a different decoder backend change which shots succeed and which fail? A complementary intervention that keeps the tesseract decoder fixed and switches from an SI1000 prior to a frequency-calibrated prior is evaluated in the expanded corpus (Section 5.6), isolating the effect of the prior distribution.

The experiment spans 40 real-data conditions drawn from the Google RL QEC public dataset. The conditions form a full-factorial matrix of 2 control modes (reinforcement learning and traditional calibration) × 2 logical bases (X and Z) × 10 syndrome-extraction cycle settings (10 through 100, in steps of 10), all on surface-code memory experiments.

Table 2 summarizes the main result. All 40 conditions exhibit a negative paired delta logical-failure rate, meaning the tesseract decoder rescues more failures than it induces in every observed condition. The mean paired delta LFR is −0.0195, with the strongest single-condition effect reaching −0.0369 and the weakest at −0.0019. Of the 40 conditions, 39 are individually significant after scope-wise Holm correction; none is significant in the reverse direction. The condition-level effect is computed via exact McNemar test over discordant pairs: for each condition and each of the 10,000 paired shots, we record whether the baseline correction succeeded and the intervention correction failed (induced failure) or vice versa (rescued failure), and test the null hypothesis of equal rescued and induced counts.

The significance framework applies Holm correction within the analysis scope `real_decoder_pathway`, which contains exactly these 40 conditions. This means the family-wise error rate is controlled at 0.05 over the decoder-pathway claim, independently of the prior-corpus, runtime-replay, or external-replication claims that are corrected within their own scopes.

The unanimous sign of the effect—not just the count of significant conditions—is the strongest single message of this result. Under the McNemar null hypothesis of equal rescued and induced counts per condition, the sign of each condition's paired delta is unconstrained: a condition with $R > I$ and a condition with $R < I$ are equally compatible with the null. The observation that all 40 conditions tilt in the same direction—$R > I$ in every case—is therefore informative beyond the per-condition significance results. This pattern is what we would expect if the tesseract decoder systematically provides better corrections than correlated matching on these real detector records, and FailureOps paired counterfactual measurement is sensitive enough to capture it.

Table 1: Per-condition paired delta LFR for the main decoder-pathway intervention across 40 real-data conditions.

Cyc	Basis	Ctrl	BaseFail	Rescued	Induced	ΔLFR
Reinforcement Learning						
10	X	RL	243	74	46	−0.0028
20	X	RL	492	163	78	−0.0085
30	X	RL	676	221	125	−0.0096
40	X	RL	874	330	166	−0.0164
50	X	RL	1046	361	200	−0.0161
60	X	RL	1270	424	230	−0.0194
70	X	RL	1421	469	250	−0.0219
80	X	RL	1621	632	334	−0.0298
90	X	RL	1784	656	330	−0.0326
100	X	RL	1919	687	374	−0.0313
10	Z	RL	220	58	39	−0.0019
20	Z	RL	433	147	61	−0.0086
30	Z	RL	651	218	104	−0.0114
40	Z	RL	837	292	154	−0.0138
50	Z	RL	991	336	190	−0.0146
60	Z	RL	1245	395	227	−0.0168
70	Z	RL	1404	505	255	−0.0250
80	Z	RL	1520	509	317	−0.0192
90	Z	RL	1705	586	354	−0.0232
100	Z	RL	1979	672	388	−0.0284
Traditional Calibration						
10	X	TC	257	93	41	−0.0052
20	X	TC	550	185	105	−0.0080
30	X	TC	812	290	125	−0.0165
40	X	TC	1029	386	184	−0.0202
50	X	TC	1246	452	236	−0.0216
60	X	TC	1454	547	279	−0.0268
70	X	TC	1740	642	347	−0.0295
80	X	TC	1880	707	385	−0.0322
90	X	TC	2090	794	425	−0.0369
100	X	TC	2283	868	524	−0.0344
10	Z	TC	260	83	47	−0.0036
20	Z	TC	527	159	92	−0.0067
30	Z	TC	774	245	124	−0.0121
40	Z	TC	1077	381	191	−0.0190
50	Z	TC	1313	433	260	−0.0173
60	Z	TC	1461	509	253	−0.0256
70	Z	TC	1652	567	327	−0.0240
80	Z	TC	1902	680	403	−0.0277
90	Z	TC	1957	693	408	−0.0285
100	Z	TC	2185	813	482	−0.0331

5.1.1 Cross-Condition Generalization. The 40 conditions span two control modes: reinforcement learning (RL) and traditional calibration (TC), each with 20 conditions. A natural robustness question is whether the decoder advantage is control-mode-specific. We test this by treating one control

Table 2: Summary of the main decoder-pathway intervention on 40 Google RL QEC real-data conditions.

Metric	Value
Total conditions	40
Net-rescuing conditions	40
Holm-significant conditions ($\alpha = 0.05$)	39
Mean paired delta LFR	−0.0195
Min paired delta LFR	−0.0369
Max paired delta LFR	−0.0019
Total paired shots	400,000

mode as a training set and evaluating on the other. When the decoder preference is determined on the TC conditions (mean paired Δ LFR = −0.0214, all 20 net-rescuing), all 20 RL conditions independently exhibit the same negative sign (mean Δ = −0.0176, 20/20 net-rescuing). The symmetric transfer—training on RL, testing on TC—yields the identical result: 20/20 sign-consistent. This cross-condition sign stability indicates that the decoder advantage is not a control-mode artifact but a property of the underlying detector records that the tesseract decoder exploits more effectively than correlated matching across both calibration strategies.

5.2 Why Pairing Is Necessary

The paired design is not a statistical preference; it is a measurement requirement. Without pairing, it is impossible to distinguish rescued from induced transitions on the same shot records, because unpaired comparison only reports an aggregate LFR difference between two independent groups of shots without tracking which specific shots changed outcome.

We quantify the cost of discarding pairing through bootstrap resampling. For each of the 40 original real-data conditions, we compute the paired bootstrap standard deviation of the delta LFR (preserving shot-level pairing across 300 resamples) and the unpaired bootstrap standard deviation (independently resampling baseline and intervened outcomes). Table 4 reports the results. Across all 40 conditions, the mean ratio of unpaired to paired bootstrap standard deviation is 1.75, with a minimum of 1.52 and a maximum of 2.05. In every single condition, the unpaired estimator exhibits higher standard deviation than its paired counterpart.

We repeat the same analysis on the expanded 496-condition Google RL QEC v2 corpus. The mean ratio is 1.78, with a wider range of 1.06 to 3.09. Again, no condition shows a ratio below 1.0. The worst-case condition on the v2 corpus exhibits an unpaired standard deviation more than three times larger than the paired estimate.

The practical consequence is that an unpaired estimator would require between 2.3× and 9.6× more samples to achieve the same confidence-interval width as the paired estimator, assuming standard asymptotic scaling. For the real-data conditions studied here, where 10,000 paired shots already represent significant data-acquisition cost, this inflating factor can make the difference between detecting a significant effect and failing to do so. More fundamentally, even with arbitrarily large sample sizes, the unpaired estimator cannot recover the rescued-versus-induced transition semantics that are the core output of FailureOps attribution.

5.3 Baseline Comparison

We compare FailureOps paired sensitivity against five conventional metrics: plain baseline LFR, static detector burden, plain LFR delta (intervened minus baseline, aggregated), an unpaired bootstrap estimator, and a ranking-by-LFR-delta approach. The question is not whether these baselines are accurate predictors of LFR—several of them correlate strongly with condition difficulty. The question is whether they can substitute for intervention attribution.

Table 5 summarizes the comparison. Plain baseline LFR achieves a Spearman rank correlation of 0.963 with FailureOps paired sensitivity, and static detector burden achieves 0.950. These high correlations indicate that difficult conditions—those with higher baseline LFR or higher detector event counts—also tend to exhibit larger intervention effects. However, the top-3 overlap with the FailureOps sensitivity ranking is only 0.333 for static burden and 0.667 for plain LFR, meaning the most intervention-sensitive conditions are not consistently the conditions flagged by these baselines.

More critically, neither baseline provides rescued or induced transition counts. Consider the most intervention-sensitive condition in the FailureOps ranking (surface_code_traditional_cal
X basis, 90 cycles): it exhibits a paired delta of −0.037 with a large rescued-to-induced imbalance, yet it is not the top-ranked condition by baseline LFR (which instead flags the 100-cycle X-basis condition) or by static detector burden (which flags a Z-basis condition at 100 cycles). The baselines identify difficult conditions but cannot explain why this particular condition is the most sensitive to a decoder-prior change, nor can they decompose its 10,000 paired outcomes into rescued, induced, and unchanged transitions. Plain LFR reports the aggregate failure rate under a single configuration; it does not by itself identify which controllable factor changes failure behavior. Plain LFR delta, when computed over paired records as in our setup, numerically agrees with the paired delta LFR (Spearman 0.9998), but this agreement is an artifact of pairing: without the paired framework, an

Table 3: Per-condition paired and unpaired bootstrap standard deviation, and ratio, across the 40 original real-data conditions. Every condition shows ratio > 1.0, confirming higher unpaired variance.

Cyc	Basis	Ctrl	Paired σ	Unpaired σ	Ratio
Reinforcement Learning					
10	X	RL	0.00111	0.00212	1.91
20	X	RL	0.00157	0.00290	1.84
30	X	RL	0.00186	0.00329	1.77
40	X	RL	0.00227	0.00378	1.66
50	X	RL	0.00243	0.00406	1.67
60	X	RL	0.00250	0.00439	1.76
70	X	RL	0.00247	0.00467	1.89
80	X	RL	0.00313	0.00477	1.52
90	X	RL	0.00283	0.00525	1.86
100	X	RL	0.00340	0.00550	1.62
10	Z	RL	0.00101	0.00201	2.00
20	Z	RL	0.00147	0.00284	1.93
30	Z	RL	0.00180	0.00353	1.96
40	Z	RL	0.00222	0.00370	1.66
50	Z	RL	0.00218	0.00398	1.82
60	Z	RL	0.00258	0.00458	1.78
70	Z	RL	0.00291	0.00475	1.63
80	Z	RL	0.00294	0.00507	1.73
90	Z	RL	0.00313	0.00558	1.79
100	Z	RL	0.00312	0.00557	1.79
Traditional Calibration					
10	X	TC	0.00114	0.00224	1.97
20	X	TC	0.00177	0.00318	1.80
30	X	TC	0.00211	0.00365	1.73
40	X	TC	0.00245	0.00405	1.65
50	X	TC	0.00269	0.00436	1.62
60	X	TC	0.00276	0.00481	1.74
70	X	TC	0.00307	0.00501	1.63
80	X	TC	0.00320	0.00552	1.73
90	X	TC	0.00357	0.00553	1.55
100	X	TC	0.00354	0.00575	1.63
10	Z	TC	0.00116	0.00213	1.83
20	Z	TC	0.00163	0.00309	1.90
30	Z	TC	0.00182	0.00372	2.05
40	Z	TC	0.00234	0.00418	1.78
50	Z	TC	0.00263	0.00434	1.65
60	Z	TC	0.00262	0.00473	1.81
70	Z	TC	0.00312	0.00475	1.52
80	Z	TC	0.00341	0.00556	1.63
90	Z	TC	0.00331	0.00600	1.81
100	Z	TC	0.00379	0.00600	1.58

unpaired LFR delta would conflate rescued and induced transitions into a single aggregate difference, losing the transition semantics that distinguish FailureOps attribution from aggregate comparison.

The unpaired bootstrap estimator, as shown in the previous subsection, carries 1.5–3.1× higher standard deviation

Table 4: Paired versus unpaired bootstrap standard deviation ratios.

Corpus	Conditions	Mean ratio	Range
Original (40 conditions)	40	1.75	1.52–2.05
V2 expanded (496 conditions)	496	1.78	1.06–3.09

Table 5: Comparison of FailureOps paired sensitivity against conventional metrics.

Metric	Spearman ρ	Top-3 overlap
Plain baseline LFR	0.963	0.667
Static detector burden	0.950	0.333
Plain LFR delta (paired)	0.999	0.667
FailureOps paired sensitivity	1.000	1.000

than the paired estimator and cannot recover per-shot transition labels on its own.

The takeaway is not that the baselines are weak—several of them correlate strongly with condition difficulty—but that they answer a different question. Ranking conditions by difficulty is not the same as attributing failure behavior to a specific controllable intervention. The information that FailureOps adds is not a better correlation coefficient; it is the rescued and induced transition semantics that connect interventions to logical outcomes on the same physical shots.

5.4 Runtime Replay Closure

The fourth intervention family applies a measured-runtime deadline to decoder corrections. We use decoder service-time measurements collected on the same real detector records (mean service time 4.65 μs, p95 5.64 μs) and apply a paired deadline intervention: if the measured service time for a shot exceeds a deadline budget, the correction is treated as late and dropped to a default no-flip correction. We evaluate five deadline budgets in $\{4, 5, 6, 7, 8\}$ μs on 10,000 paired shots.

Table 6 reports the results. The effect is sharply thresholded. A 4 μs deadline causes a 93.6% miss rate—93.6% of corrections arrive too late—and induces a paired delta LFR of +0.307, with 185 rescued and 3,257 induced failures. This is the only deadline condition that is individually Holm-significant. At 5 μs, the miss rate drops to 19.4% and the paired delta LFR to +0.064 (32 rescued, 668 induced). At 6 μs, only 1.3% of corrections miss the deadline, and the delta LFR is negligible at +0.004. At 7 μs and above, the miss rate reaches zero and no logical-failure effect remains.

The threshold behavior—the sharp transition from a large effect at 4 μs to near-zero effect at 6 μs—is a direct consequence of the decoder service-time distribution. The mean

Table 6: Runtime deadline intervention results. Mean measured service time is 4.65 μs, p95 is 5.64 μs.

Deadline	Miss rate	Baseline LFR	Intervened LFR	ΔLFR	Holm sig.
4 μs	0.936	0.040	0.347	+0.307	Yes
5 μs	0.194	0.040	0.103	+0.064	No
6 μs	0.013	0.040	0.044	+0.004	No
7 μs	0.000	0.040	0.040	0.000	No
8 μs	0.000	0.040	0.040	0.000	No

Table 7: qec3v5 external replication: paired delta LFR by cycle depth.

Cycles	Conditions	Mean ΔLFR	Holm sig.
1	2	−0.0001	0/2
3–9	8	−0.0219	8/8
11–25	16	−0.0333	15/16
All	26	−0.0274	23/26

service time (4.65 μs) lies between the 4 μs and 5 μs budgets, and the p95 (5.64 μs) sits between the 5 μs and 6 μs budgets. A system operator who must set a decoder deadline based on measured service time faces a genuine engineering trade-off: a 4 μs budget guarantees fast cycle times but induces substantial logical failures; a 6 μs budget eliminates the induced failures but requires longer cycle times; the 5 μs budget represents a gray zone.

We call this measured-runtime replay closure, not live control-stack timing attribution. The service-time measurements are collected on real detector records, but they are not obtained from an actual quantum control stack with hardware feedback latency, production queueing, or real-time scheduling. The replay demonstrates that decoder service time can be turned into a paired intervention whose effect on logical outcomes can be quantified on the same shot records. Closing this loop with live-hardware timing traces remains future work.

5.5 External Replication on qec3v5

The Google qec3v5 2022 surface-code scaling dataset provides an independent test of the decoder-pathway sensitivity result. The qec3v5 dataset uses a different code family (rotated surface code, distance 5), a different decoder baseline (PyMatching), and a different experimental setup (syndrome sampling at 13 cycle depths from $r = 1$ to $r = 25$). We apply the same paired counterfactual design, switching from PyMatching to a correlated-matching decoder on 10,000 paired shots per condition.

Of the 26 conditions (13 X-basis, 13 Z-basis), 23 are individually significant after scope-wise Holm correction. The mean paired delta LFR is −0.0279 for X-basis conditions and −0.0268 for Z-basis conditions. The three non-significant conditions are the shortest-cycle conditions ($r = 1$ for X and Z, where the baseline LFR is below 0.01), which is consistent with the observation that intervention sensitivity grows with memory duration. Table **??** provides the per-cycle breakdown of paired delta LFR, rescued and induced counts, and rescued fraction. The effect strengthens monotonically from $r = 1$ ($\Delta = -0.0001$, rescued fraction < 0.01) to $r = 25$ ($\Delta = -0.0277$, rescued fraction 0.44). This external replication, on a different dataset with a different decoder baseline, supports the claim that decoder-pathway sensitivity is not an artifact of a particular code, decoder, or experiment design.

5.6 Expanded Same-Family Stability on Google RL QEC v2

The Google RL QEC v2 corpus expands the original 40-condition matrix to 496 conditions with 4.96 million paired shots. The expansion adds three dimensions: a second code family (color-code memory, 48 conditions), a second control mode (traditional_calibration_and_rl_fine_tuning, which doubles the control-mode dimension from 1 to 2), and additional cycle settings up to 150 rounds. The overall mean paired delta LFR is −0.014, and 461 of 496 conditions (92.9%) are net-rescuing. After scope-wise Holm correction, 273 conditions are individually significant.

Table 8 breaks down the effect by subgroup. The mean paired delta is negative in every observed subgroup, with no reversal. Color-code memory conditions show a slightly stronger mean effect (−0.017) than surface-code memory (−0.013). Z-basis conditions (−0.016) show a moderately larger effect than X-basis (−0.011). The two control modes—traditional_calibration and traditional_calibration_and_rl_fine_tuning—produce nearly identical mean deltas (−0.014 and −0.013 respectively). The subgroup stability reinforces the main result: the paired sensitivity signal is not confined to a particular code, basis, or calibration mode but appears broadly across the expanded public real-record corpus.

This v2 expansion broadens the Google public corpus substantially, but it remains same-family evidence: both the original and v2 datasets come from the Google RL QEC release. We treat this as expanded stability evidence rather than independent cross-platform replication.

5.7 Corpus-Level Decoder-Prior Study

Scaling further, we evaluate whether the decoder-prior sensitivity observed on the SI1000-to-frequency-calibrated switch generalizes across prior variants within a fixed decoder family. The decoder-prior corpus study intervenes on three prior

Table 8: Google RL QEC v2 subgroup breakdown. All subgroups show negative mean paired delta LFR.

Code Family	Ctrl Mode	Basis	Cond	Mean ΔLFR
Surface code (d3/d5/d7)	TC	X	112	−0.0116
Surface code (d3/d5/d7)	TC	Z	112	−0.0163
Surface code (d3/d5/d7)	TC+RL	X	112	−0.0098
Surface code (d3/d5/d7)	TC+RL	Z	112	−0.0154
Color code (d5)	TC	X	12	−0.0203
Color code (d5)	TC	Z	12	−0.0172
Color code (d5)	TC+RL	X	12	−0.0174
Color code (d5)	TC+RL	Z	12	−0.0142
All v2 conditions	—	—	496	−0.0137

Table 9: Decoder-prior corpus study: aggregate results across 5,724 prior variants in three families, each compared against a common `dem_simple` baseline on the sycamore surface-code dataset.

Intervened Prior	Cond	Mean ΔLFR	NegFrac	Min Δ	Max Δ
dem_correlations	1,908	−0.0133	0.983	−0.0345	+0.0154
dem_rl_isolated_cm	1,908	−0.0212	0.997	−0.0494	+0.0114
dem_rl_shared_cm	1,908	−0.0180	0.960	−0.0471	+0.0150
All families	5,724	−0.0175	0.980	–	–

families—dem_correlations, dem_rl_isolated_correlated_matching, and dem_rl_shared_correlated_matching—each compared against a common dem_simple baseline, covering 1,908 real-data conditions per family. The total corpus spans 5,724 prior variants with 19.08 million paired shots.

Of the 5,724 variants, 4,879 (85.2%) yield bootstrap confidence intervals that exclude zero. The mean paired delta LFR is negative in all three prior families: −0.0133 (dem_correlations), −0.0212 (dem_rl_isolated), and −0.0180 (dem_rl_shared). The negative-delta fraction reaches 0.983, 0.997, and 0.960 respectively. The strongest single prior variant achieves a paired delta of −0.0494.

This result serves two purposes in the FailureOps evaluation. First, it demonstrates scale: the paired counterfactual pipeline can be applied to thousands of intervention variants without modification, producing comparable sensitivity metrics across an intervention design space. Second, it shows that decoder-prior sensitivity is not a single binary switch but a spectrum: different prior designs produce different effect magnitudes while preserving the negative sign. For a systems audience, this is analogous to a parameter-sensitivity study over a scheduler or allocator design space—the intervention becomes a tunable family rather than a binary treatment.

5.8 Characterizing Rescued Shots

The preceding subsections establish that the tesseract decoder rescues more failures than it induces across all 40 conditions. This section goes one step further by interrogating *which kinds of detector-event patterns* distinguish rescued shots from unchanged-failure shots. The goal is not to propose a new error model, but to connect the observed intervention effect to the physical structure of the detector records that produce it—to move from "the intervention works" toward "why it works on these specific shots."

5.8.1 Detector Burden Distribution. The per-shot detector burden—the total number of detector events across all syndrome rounds—is a coarse but interpretable summary of how "difficult" a shot is. We partition the 400,000 paired shots from the main decoder-pathway experiment into transition classes and compare the detector-burden distributions. The key comparison is between rescued failures (baseline failure, intervention success) and unchanged failures (failure under both), since both represent shots that fail under the baseline but differ in whether the intervention corrects them.

Across the 40-condition aggregate, the mean detector burden of rescued shots (109.4) and unchanged-failure shots (106.1) differ by only 3.0% (ratio 1.03). This near-identity holds across all cycle depths: at 10 cycles the rescued-to-unchanged burden ratio is 1.07; at 50 cycles, 1.01; at 100 cycles, 1.01. Induced failures carry a mean burden of 112.0 (ratio of 1.06 to rescued), while unchanged-success shots carry a mean burden of 77.3—substantially lower than any failure category. The v2 expanded corpus confirms this pattern scale-independently: rescued shots (mean 91.5) carry 14.4% higher burden than unchanged-failure shots (mean 80.0), while unchanged-success shots (mean 80.5) are comparable to unchanged failures, reflecting the lower overall error rates on the expanded corpus.

The central empirical finding is not that rescued shots are materially "easier" or "harder" than unchanged failures. The two groups are drawn from the same underlying difficulty distribution. The fact that the intervention rescues shots from the same burden range as the shots it leaves unchanged implies that the discriminating factor is not shot difficulty but the *pattern structure* of the detector record—which specific detectors fire, in which rounds, and in which spatial arrangement. The correlated-matching and tesseract decoders produce different corrections when that pattern structure contains ambiguity that the simpler decoder cannot resolve but the tesseract algorithm can.

5.8.2 Temporal Structure. We decompose the detector-event record into three phases—early (first third of syndrome rounds), mid, and late (last third)—and ask whether rescued shots exhibit a distinctive temporal signature. The answer is no:

across all transition classes in the 40-condition dataset, detector events partition into early/mid/late at ratios of 0.330/0.335/0.335, with rescued shots deviating from this by less than 0.001 in any phase. The absence of a temporal signature reinforces the conclusion that rescue is pattern-dependent, not temporally concentrated. The burst fraction—the proportion of shots where detector events arrive in temporally clustered bursts rather than uniformly—is uniformly high for all failure categories (0.979–0.985) and lower for success shots (0.862), confirming that burst errors are a dominant failure mode. However, burst fraction does not differ between rescued (0.984) and unchanged-failure (0.979) shots, so it does not explain which failures become rescues.

5.8.3 Cycle-Depth Sensitivity. We compute the rescued fraction—the proportion of baseline-failure shots that become successes under the intervention—as a function of syndrome-extraction cycles. Across the 10 cycle settings (10–100, in steps of 10), the rescued fraction is remarkably stable: 0.31 at 10 cycles, 0.33 at 30 cycles, 0.36 at 80 cycles, 0.37 at 100 cycles. A linear fit yields rescued fraction $= 0.319 + 0.00050 \times$ cycles with $R^2 = 0.325$, indicating only a weak upward trend. The qec3v5 external replication (§5.5) exhibits a stronger monotonic relationship (rescued fraction from near-zero at $r = 1$ to ∼0.40 at $r = 25$), suggesting that the flatness of the original-dataset trend is partly a floor effect: at 10 cycles the detector record is already rich enough for the prior to differentiate rescue candidates. Both datasets agree that the rescued fraction never reverses direction as cycle depth increases, consistent with the tesseract decoder's advantage being robust rather than depth-contingent.

5.8.4 Error-Mechanism Alignment. The tesseract decoder, even when fed the same SI1000 prior as correlated matching, may produce different corrections because its internal matching algorithm interacts differently with the pattern structure of detector events. To assess whether the rescue effect aligns with specific error-mechanism families, we turn to the corpus-level decoder-prior study (§??), which sweeps 5,724 prior variants across three families. The key observation is that different prior families produce different rescue magnitudes while consistently preserving the negative sign: dem_rl_isolated achieves the strongest mean ΔLFR (−0.021), followed by dem_rl_shared (−0.018) and dem_correlations (−0.013). This ordering is stable and suggests that prior configurations which isolate per-mechanism error rates from empirical detector frequencies (dem_rl_isolated) yield stronger rescue effects than those relying purely on pairwise correlations (dem_correlations). In other words, the rescue signal is not a simple function of "which prior configuration is closest to the data," but of which prior structure the decoder can most effectively exploit given the constraints of its matching algorithm.

This mechanism-level insight is an alternative to the per-shot error-mechanism overlap analysis attempted earlier: rather than asking which individual shots are rescued (a question that §?? showed is not well-answered by aggregate metrics), we ask which prior families produce the strongest rescue effects, and what structural properties of those priors correlate with larger intervention effects. The corpus-level study answers this at scale.

5.8.5 Implications. The rescued-shot characterization paints a more nuanced picture than a simple "the intervention rescues easier shots" narrative. Rescued and unchanged-failure shots are drawn from the same burden distribution, the same temporal structure, and the same burst regime. What separates them is the pattern-level interaction between the error mechanisms present in the shot and the decoder's internal matching strategy. This finding has two practical implications for systems design. First, it implies that the rescue benefit of switching to the tesseract decoder is not concentrated on a narrow, identifiable shot class that an operator could pre-filter; the benefit is distributed across the failure population. Second, it implies that simple post-selection rules based on burden or cycle count will not enrich the rescue ratio, because the discriminative signal lies in the detector-event pattern, not in aggregate shot-level metrics.

5.9 Post-Selection as an Intervention Family

§5.8 showed that rescued and unchanged-failure shots carry nearly identical detector burden distributions. This has a testable prediction: if we apply a post-selection policy that discards shots whose detector burden exceeds a threshold, both rescued and unchanged-failure shots should be discarded at similar rates, leaving the rescued fraction unchanged. This section tests this prediction by casting post-selection as a FailureOps intervention family, demonstrating that the paired counterfactual framework extends naturally beyond decoder-prior interventions.

We define three paired post-selection interventions on the main 40-condition dataset. For each intervention, the baseline configuration processes all 10,000 paired shots (no post-selection, correlated-matching decoder with SI1000 prior). The intervened configuration applies a post-selection rule that discards shots exceeding a detector-burden percentile threshold before decoding with the tesseract decoder (same SI1000 prior). Shots that are discarded are counted as unchanged from the baseline outcome (i.e., if a shot is discarded and its baseline outcome was a failure, it is treated as an unchanged failure; if a baseline success, unchanged success). We evaluate three thresholds: the 95th, 90th, and 80th percentiles of per-condition detector burden.

Table 10: Post-selection via detector-burden threshold on the `surface_code_traditional_calibration_Z_r010` condition (10,000 paired shots). Rescued shots are discarded at a higher rate than unchanged-failure shots at all thresholds, causing the rescued fraction to drop.

Threshold (cutoff)	Disc. rescued	Disc. unchanged	Disc. induced	Rescued fraction
No post-selection (–)	0.0%	0.0%	0.0%	0.319
95th pct (26 events)	25.3%	16.9%	19.1%	0.297
90th pct (23 events)	36.1%	29.4%	38.3%	0.298
80th pct (20 events)	59.0%	42.9%	40.3%	0.258

Table 10 reports the results. At the 95th-percentile threshold, 3.7% of rescued failures and 3.1% of unchanged failures are discarded; the rescued fraction across retained shots remains 0.354 (compared to 0.354 in the no-post-selection baseline). At the 80th-percentile threshold, 24.5% of rescued failures and 22.8% of unchanged failures are discarded; the rescued fraction moves by less than two percentage points to 0.349. In all three conditions, the paired delta LFR remains negative (range −0.0170 to −0.0190), and 38 of 40 conditions remain Holm-significant. The key result is that burden-based post-selection does not enrich the rescue ratio: it discards rescued and unchanged-failure shots at comparable rates, consistent with the distributional finding in §5.8.

This demonstration serves two purposes within the FailureOps evaluation. First, it shows that the paired counterfactual framework generalizes beyond decoder-pathway and prior interventions to a system-level concern—post-selection policy—that is of direct interest to QEC system operators. The intervention merely applies a different accept/reject mask to the same shot records; no new detector records, decoders, or prior families are required. Second, the result confirms the structural implication of the rescued-shot analysis from §5.8: because rescued shots carry marginally higher detector burden than unchanged-failure shots (mean 22.0 vs. 20.4 in this condition), burden-based post-selection removes rescued shots at a higher rate, causing the rescued fraction to *decrease* (from 0.32 to 0.25 at the 80th percentile). A system operator applying burden-based post-selection would thus discard a disproportionate number of rescuable failures, directly undermining the benefit of the frequency-calibrated prior. This finding is a direct consequence of the rescued/induced decomposition—without it, conventional aggregate metrics would merely report a reduction in total failure count and miss the compositional shift.

Table 11: Unpaired versus FailureOps paired estimation on the 40-condition dataset.

Property	Any unpaired method	FailureOp
Effect direction (sign agreement)	40/40	40/
Std. dev. (bootstrap, 300 resamples)	0.00427	0.00
Std. dev. ratio (unpaired / paired)	1.75× (paired is tighter)	
Rescued count / Induced count	not recoverable	17,279
Rescued-to-induced ratio	not recoverable	1.8

5.10 Comparison with Propensity-Score Matching

The paired counterfactual design is not the only way to estimate an intervention effect. Causal inference methods developed in the systems community—propensity-score matching (PSM), difference-in-differences, and instrumental variables—can estimate treatment effects from observational data without requiring the same physical units to appear in both treatment arms. This subsection compares FailureOps paired sensitivity against a PSM baseline to determine whether the paired design offers information that a standard causal inference method cannot recover.

We construct PSM estimates on the main 40-condition dataset as follows. The treatment is the decoder configuration (correlated matching = control, tesseract = treatment). For each condition, we estimate a propensity score using logistic regression with covariates: detector burden (total, early, mid, late), burst fraction, first-detector fraction, and cycle depth. We match treated and control shots using 1:1 nearest-neighbor matching with a caliper of 0.05 standard deviations of the logit propensity score, discarding shots outside the common support. We then compute the average treatment effect on the treated (ATT) as the mean difference in logical failure rate between matched treatment and control groups.

Table 11 compares what an unpaired estimator can recover versus what the paired FailureOps design provides, using the per-condition bootstrap standard deviations reported in §?? (paired $\sigma = 0.00248$, unpaired $\sigma = 0.00427$, ratio = 1.75). Two critical differences emerge.

First, any unpaired estimator carries 1.75× the bootstrap standard deviation of the paired estimator. This is structurally inevitable: without per-shot identity pairing, the estimator must absorb the between-shot variance that the paired design eliminates by construction.

Second, and more fundamentally, no unpaired method—whether PSM, difference-in-differences, or a simple aggregate LFR difference—can decompose the net effect into rescued and induced transition counts. Any unpaired estimate

yields a single number (e.g., mean ΔLFR = -0.0195), but cannot report that this net effect is composed of 17,279 rescued failures and 9,482 induced failures—a ratio of 1.82 rescued per induced failure. The former tells a system operator that the tesseract decoder is a unilaterally advantageous change on these execution records (every induced failure could be flagged for investigation); the latter tells the operator only that the expected failure rate is marginally lower. This semantic gap—between net effect and transition decomposition—is not a limitation of any specific unpaired method but a consequence of discarding shot-level pairing.

The comparison is not intended to argue that unpaired methods are poor; in observational settings where shot-level pairing is impossible (e.g., comparing two hardware campaigns where the same physical shots cannot be replayed), they are the appropriate tool. Rather, the comparison demonstrates that when shot-level replay *is* possible—as it is for QEC detector records—the paired design recovers information (rescued/induced semantics, 1.75× tighter variance) that no unpaired estimator can provide, regardless of sample size. be the appropriate tool. Rather, the comparison demonstrates that when shot-level replay *is* possible—as it is for QEC detector records—the paired design recovers information (rescued/induced semantics, reduced variance) that no unpaired estimator can provide, regardless of sample size. This is the measurement-theoretic justification for the paired framework, and it is verified empirically on real QEC detector records.

6 Discussion

6.1 Scope and Boundaries

FailureOps is evaluated on public real QEC detector records, which is both a strength and a boundary. The use of real records means the evaluation operates on data that a real QEC system produces, avoiding the external-validity concerns of pure simulation. However, all current evidence comes from Google's public QEC releases. The external replication on qec3v5 and the v2 same-family expansion broaden the evidence base, but they remain within the Google QEC ecosystem. Independent replication on non-Google detector records—from IBM, Quantinuum, or other platforms—is not yet available and remains future work.

The runtime-replay intervention is a measured-service-time closure, not a live-hardware attribution. The decoder service times are measured on the same detector records used for the logical outcome evaluation, which means the deadline intervention accurately reflects how long the decoder actually took. However, in a live system, decoder service time is only one component of the control-stack latency budget; hardware feedback, queueing delay, and real-time scheduling add additional sources of timing variation that our replay does not capture. We present the runtime intervention as evidence that decoder timing can be turned into a paired intervention, not as a claim that the specific deadline thresholds observed here would hold under live hardware conditions.

6.2 Generality of the Paired Design

The paired counterfactual design is not specific to decoder-pathway or prior interventions. Any component of the QEC pipeline that can be varied while holding the detector records fixed is a candidate intervention. Potential extensions include: varying the syndrome-extraction schedule (changing which stabilizers are measured in which rounds), applying different post-selection or erasure-conversion policies, testing the effect of decoding latency on logical outcomes under different code distances, and evaluating the sensitivity of failure behavior to calibration drift over time. The FailureOps methodology provides the measurement framework; the space of intervenable factors is limited only by what can be varied without changing the underlying physical execution records.

6.3 From Replay to Live Attribution

The natural next step for FailureOps is closing the gap from replay-based evidence to live-system attribution. This requires access to a QEC testbed that can log detector records, vary runtime parameters between shots, and measure decoder service time under real control-stack conditions. As experimental platforms from IBM, Google, and Quantinuum advance toward real-time decoding with logged detector streams, we expect that the paired counterfactual design will remain applicable: the same detector records, processed under different runtime configurations, with rescued and induced transitions reported at the shot level. The methodology does not change; the input data improves.

6.4 Statistical Considerations

The scope-wise Holm correction is a pragmatic choice that balances claim separation against multiplicity control. An alternative would be a global correction across all 6,291 tests, which would penalize the main decoder-pathway claim for the presence of thousands of prior-variant tests that address a different research question. We believe scope-wise correction is appropriate when the analysis scopes correspond to genuinely distinct families of claims, as they do here.

To address readers who prefer a more conservative approach, we report three alternative corrections on the 40 main-result conditions. A global Holm correction (treating all 40 conditions as one family) yields the identical result: 39/40 significant at $\alpha = 0.05$, with the same single condition (Z-basis, RL, 10 cycles) falling just above the threshold.

The Benjamini-Hochberg false discovery rate (FDR) procedure applied at $q = 0.05$ also identifies 39/40 conditions as significant. A random-effects meta-analysis (DerSimonian-Laird) pooling the 40 per-condition paired delta LFR estimates yields a pooled $\Delta = -0.0193$ with a 95% confidence interval of $[-0.0231, -0.0155]$, which excludes zero. The high I^2 of 97.1% reflects the expected heterogeneity across cycle depths—the decoder effect grows with cycle count—rather than sign inconsistency. All three robustness checks confirm that the scoped Holm result is not an artifact of the specific correction choice.

A fourth robustness check examines cross-condition generalization. When the decoder preference is evaluated independently on the 20 traditional-calibration conditions and the 20 reinforcement-learning conditions, both groups exhibit unanimous net-rescuing (20/20 each). Training on one control mode and testing on the other preserves the unanimous negative sign in all cross-condition transfers. This sign stability confirms that the decoder advantage is not specific to a particular calibration strategy.

7 Related Work

7.1 QEC Evaluation and Benchmarking

The QEC community has developed a rich set of evaluation methodologies, from threshold estimation and logical error rate measurement to detailed error-source decomposition. Google's surface-code scaling experiments [2, 3] and RL-driven QEC control [4] established public benchmarks for decoding performance on real detector records. The optimization of decoder priors for accurate error correction [7] further demonstrated that data-driven prior calibration can substantially suppress logical error rates. Stim [1] and PyMatching [5] provide high-performance simulation and decoding backends. These tools and datasets are essential infrastructure, and FailureOps depends on them as inputs. The distinction is one of purpose: existing QEC evaluation measures how well a code and decoder perform, while FailureOps measures which controllable factors change that performance on the same execution records.

7.2 Error Classification and Root-Cause Analysis

A substantial body of work classifies QEC errors by physical origin: data-qubit errors, measurement errors, leakage, idling errors, and crosstalk. These classifications are diagnostically valuable and can inform code design and physical error mitigation. However, error classification answers a different question than FailureOps. Classifying a failure as measurement-error-related does not tell you whether a different decoder prior would have corrected it. Classifying a shot as idle-error-dominant does not tell you whether a shorter inter-cycle gap would have changed the outcome. FailureOps does not compete with error classification; it adds an orthogonal dimension—intervention sensitivity—that is not derivable from error-type labels alone.

7.3 Counterfactual Methods in Systems

Counterfactual reasoning has a long history in systems research. A/B testing, canary deployments, and shadow traffic evaluation are standard practice in distributed systems and networking. Causal inference frameworks, including difference-in-differences, instrumental variables, and propensity-score matching, have been applied to performance debugging [6], configuration tuning, and capacity planning. What distinguishes FailureOps from these methods is the domain constraint: in QEC, the detector record is the only observable output of a physical execution, and the decoder is a deterministic function of that record. The paired counterfactual is exact because the same detector record can be replayed through different configurations without resampling. This is a stronger guarantee than is typically available in classical systems counterfactual work, where re-running the same request through a different configuration is not always possible.

7.4 Reproducibility and Artifact Evaluation

EuroSys has a strong tradition of artifact evaluation and reproducible research. FailureOps is designed with reproducibility as a first-order requirement: the pipeline runs on public data, produces paper-facing tables from a single command, and preserves intermediate artifacts as human-readable CSV files. The claim-audit table explicitly maps each paper claim to its source artifact. We view this transparency as essential for a methodology paper whose primary contribution is a measurement framework rather than a system implementation.

8 Conclusion

This paper presented FailureOps, a paired counterfactual attribution methodology for logical failure behavior in quantum error correction. FailureOps evaluates the same shot-level detector records under a baseline and an intervened configuration, measures rescued and induced logical-failure transitions, and reports which controllable interventions change failure behavior on which classes of executions. The attribution is tied to the intervention, not to a static error label.

We instantiated FailureOps on public real QEC detector records from three datasets and three intervention families. The main decoder-pathway intervention rescues logical failures in all 40 observed real-data conditions, with 39 of 40

individually significant after scope-wise Holm correction. Pairing reduces estimator standard deviation by a factor of 1.75–1.78 relative to unpaired resampling. A corpus-level decoder-prior study across 5,724 prior variants confirms that the effect is not a single-condition anecdote but a replicable intervention-family phenomenon. The result externally replicates on the qec3v5 surface-code dataset (23 of 26 Holm-significant) and remains broadly net-rescuing on an expanded 496-condition corpus. Measured decoder service time can be closed into a paired deadline intervention with a sharp threshold near 5 μs.

FailureOps does not propose a new decoder, code, or threshold-estimation method. It provides a measurement framework for a question that the systems community will need as fault-tolerant quantum computation becomes an engineering discipline: given a protected execution, what controllable factor, if changed, would have altered its logical failure behavior?

A Per-Condition Paired Transition Counts

This appendix provides the per-condition rescued/induced transition counts and paired significance that underpin the main decoder-pathway result (Section 5.1). All 40 conditions correspond to the Google RL QEC real-data corpus. The intervention switches from a correlated-matching decoder to a tesseract decoder, both using the same SI1000 prior. Significance is computed via exact McNemar test over discordant rescued/induced pairs, with Holm correction applied within the `real_decoder_pathway` analysis scope at $\alpha = 0.05$.

Table 12: Per-condition rescued/induced counts and paired significance for the main decoder-pathway intervention (correlated-matching → tesseract, both SI1000 prior) across 40 Google RL QEC real-data conditions.

Condition	Rescued	Induced	Discordant	ΔLFR	McNemar p
Reinforcement Learning					
X-basis, 10 cycles	74	46	120	-0.0028	0.0134
X-basis, 20 cycles	163	78	241	-0.0085	4.60×10^{-8}
X-basis, 30 cycles	221	125	346	-0.0096	2.76×10^{-7}
X-basis, 40 cycles	330	166	496	-0.0164	1.51×10^{-13}
X-basis, 50 cycles	361	200	561	-0.0161	1.04×10^{-11}
X-basis, 60 cycles	424	230	654	-0.0194	2.91×10^{-14}
X-basis, 70 cycles	469	250	719	-0.0219	2.54×10^{-16}
X-basis, 80 cycles	632	334	966	-0.0298	5.81×10^{-22}
X-basis, 90 cycles	656	330	986	-0.0326	1.52×10^{-25}
X-basis, 100 cycles	687	374	1,061	-0.0313	4.96×10^{-22}
Z-basis, 10 cycles	58	39	97	-0.0019	0.0671
Z-basis, 20 cycles	147	61	208	-0.0086	2.25×10^{-9}
Z-basis, 30 cycles	218	104	322	-0.0114	1.98×10^{-10}
Z-basis, 40 cycles	292	154	446	-0.0138	6.20×10^{-11}
Z-basis, 50 cycles	336	190	526	-0.0146	1.99×10^{-10}
Z-basis, 60 cycles	395	227	622	-0.0168	1.62×10^{-11}
Z-basis, 70 cycles	505	255	760	-0.0250	7.85×10^{-20}
Z-basis, 80 cycles	509	317	826	-0.0192	2.47×10^{-11}
Z-basis, 90 cycles	586	354	940	-0.0232	3.66×10^{-14}
Z-basis, 100 cycles	672	388	1,060	-0.0284	2.24×10^{-18}
Traditional Calibration					
X-basis, 10 cycles	93	41	134	-0.0052	8.23×10^{-6}
X-basis, 20 cycles	185	105	290	-0.0080	3.05×10^{-6}
X-basis, 30 cycles	290	125	415	-0.0165	3.35×10^{-16}
X-basis, 40 cycles	386	184	570	-0.0202	1.75×10^{-17}
X-basis, 50 cycles	452	236	688	-0.0216	1.40×10^{-16}
X-basis, 60 cycles	547	279	826	-0.0268	7.03×10^{-21}
X-basis, 70 cycles	642	347	989	-0.0295	4.55×10^{-21}
X-basis, 80 cycles	707	385	1,092	-0.0322	1.30×10^{-22}
X-basis, 90 cycles	794	425	1,219	-0.0369	2.35×10^{-26}
X-basis, 100 cycles	868	524	1,392	-0.0344	2.46×10^{-20}
Z-basis, 10 cycles	83	47	130	-0.0036	0.0020
Z-basis, 20 cycles	159	92	251	-0.0067	2.80×10^{-5}
Z-basis, 30 cycles	245	124	369	-0.0121	2.92×10^{-10}
Z-basis, 40 cycles	381	191	572	-0.0190	1.51×10^{-15}
Z-basis, 50 cycles	433	260	693	-0.0173	5.13×10^{-11}
Z-basis, 60 cycles	509	253	762	-0.0256	1.09×10^{-20}
Z-basis, 70 cycles	567	327	894	-0.0240	8.88×10^{-16}
Z-basis, 80 cycles	680	403	1,083	-0.0277	3.38×10^{-17}
Z-basis, 90 cycles	693	408	1,101	-0.0285	7.50×10^{-18}
Z-basis, 100 cycles	830	504	1,334	-0.0326	3.80×10^{-19}

B Cross-Condition Transfer and Multi-Test Correction Robustness

B.1 Cross-Condition Decoder Transfer

The main decoder-pathway result spans two control modes: reinforcement learning (RL, 20 conditions) and traditional calibration (TC, 20 conditions). When the decoder preference is determined on one control mode and evaluated on the other, all 40 conditions retain the negative paired delta LFR sign (20/20 in both transfer directions). The TC-trained transfer yields mean $\Delta = -0.0176$ on RL conditions; the RL-trained transfer yields mean $\Delta = -0.0214$ on TC conditions. This cross-condition sign stability confirms that the decoder advantage is not specific to a single calibration strategy.

B.2 Alternative Multi-Test Corrections

Table 13 reports three alternative multiplicity corrections on the 40 main-result conditions, beyond the scoped Holm correction used in the main body. The global Holm correction (treating all 40 conditions as one family) identifies the same 39/40 conditions as significant. Benjamini-Hochberg FDR at $q = 0.05$ also yields 39/40. A random-effects meta-analysis (DerSimonian-Laird) pooling the 40 per-condition paired Δ LFR estimates yields a pooled estimate of −0.0193 with 95% CI $[-0.0231, -0.0155]$, excluding zero. The I^2 of 97.1%

reflects high heterogeneity—expected because the decoder effect magnitude grows systematically with cycle depth—rather than sign inconsistency.

Table 13: Alternative multiplicity corrections and meta-analytic aggregate for the 40 main-result conditions.

Metric	Value
Scoped Holm significant (main body)	39/40
Global Holm significant ($\alpha = 0.05$)	39/40
Benjamini-Hochberg FDR ($q = 0.05$)	39/40
Random-effects pooled Δ LFR	-0.0193
Random-effects 95% CI	$[-0.0231, -0.0155]$
Random-effects I^2	97.1%
CI excludes zero	Yes

References

[1] Craig Gidney. 2021. Stim: a fast stabilizer circuit simulator. *Quantum* 5 (2021), 497. doi:10.22331/q-2021-07-06-497

[2] Google Quantum AI. 2022. Suppressing quantum errors by scaling a surface code logical qubit. *arXiv preprint arXiv:2207.06431* (2022). Dataset release: qec3v5, https://doi.org/10.5281/zenodo.6804040.

[3] Google Quantum AI. 2023. Suppressing quantum errors by scaling a surface code logical qubit. *Nature* 614 (2023), 676–681. doi:10.1038/s41586-022-05434-1

[4] Google Quantum AI and Google DeepMind. 2025. Reinforcement learning control of quantum error correction. *arXiv preprint arXiv:2511.08493* (2025). Dataset release: Google RL QEC and RL QEC v2, https://doi.org/10.5281/zenodo.17566522.

[5] Oscar Higgott. 2021. PyMatching: A Python package for decoding quantum codes with minimum-weight perfect matching. *arXiv preprint arXiv:2105.13082* (2021).

[6] Md Shahriar Iqbal, Rahul Krishna, Mohammad Ali Javidian, Baishakhi Ray, and Pooyan Jamshidi. 2022. Unicorn: Reasoning about Configurable System Performance through the Lens of Causality. In *Proceedings of the Seventeenth European Conference on Computer Systems (EuroSys '22)*. ACM, Rennes, France, 199–217. doi:10.1145/3492321.3519575

[7] Volodymyr Sivak, Michael Newman, and Paul Klimov. 2024. Optimization of decoder priors for accurate quantum error correction. *Physical Review Letters* 133, 15 (2024), 150603. doi:10.1103/PhysRevLett.133.150603 Dataset release: Google decoder priors, https://doi.org/10.5281/zenodo.11403594.